BayesianEstimationTool Manual

BayesianEstimationTool (BET) helps you fit a mechanistic model to a population of related individuals and then use what you learned to predict outcomes for a new individual. An *individual* (or *entry*) might be a patient, experiment, or replicate. You supply the scientific model and how observations arise; BET handles the repeated work of hierarchical Bayesian inference, population-to-individual transfer, and posterior prediction.

A mathematical model describes how a system changes or behaves. For example, a simple tumor growth model can describe how tumor size changes over time using a growth-rate parameter: a larger rate produces faster growth. If tumor sizes are measured repeatedly for several patients, the model can be compared with each patient's measurements to estimate how quickly their tumors grow. The same equations can describe every patient, even though the parameter values may differ.

Rather than estimating each patient's growth rate independently, BET uses a **hierarchical model** to learn how growth rates vary across the patient population. This allows information from patients with more measurements to inform estimates for patients with less data. For a new patient, BET can generate predictions from the learned population distribution before any tumor measurements are available, or use the patient's available measurements to update their individual growth-rate estimate. These are **population prediction** and **observation-conditioned prediction**, respectively. The distinction is central to BET's training and testing workflow.

The workflow at a glance

1. **Describe the data.** Store observations and covariates for each entry in arrays.
2. **Define the mechanistic model.** Write a function that predicts outputs for one entry from shared and individual parameters.
3. **Define uncertainty.** Supply priors for shared parameters, names for individual latent effects, and a likelihood describing measurement noise.
4. **Train.** Infer population parameters and the distribution of individual effects from training entries.
5. **Test.** Predict for unseen entries, optionally conditioning on their available observations.
6. **Score and inspect.** Compare predictions with held-out data and examine posterior uncertainty.

Core components

Component	Purpose	You provide
`data`	Inputs for all entries	Dictionary of JAX-compatible arrays
`ModelFn`	Scientific model for one entry	Function returning named predictions
`GlobalPriorFn`	Prior distributions for shared unknowns	Function returning distributions
`localZNames`	Names of individual-varying standardized effects	List of strings
`TrainLikelihoodFn`	How training observations relate to predictions	Function returning observations and distributions
`Estimator`	Coordinates the model and inference state	Model, data, and optional prior/effect definitions
`Train` / `Test`	Population fitting / new-entry prediction	Indices, likelihood, and inference method

Component	Purpose	You provide
`ScoreFn`	Defines a prediction quality measure	Function returning a score per entry

Notation used throughout: `[]` denotes a list or array (for example, `[0, 1]`); `{}` denotes a dictionary mapping names to values (for example, `{"time": times}`); `(a, b)` denotes a tuple; and `[N, T]` describes an array shape with `N` entries and `T` time positions. These symbols indicate Python data structures, not additional BET syntax.

Part 1: Introduction and core concepts

Overview

BET separates the scientific model from hierarchical inference. You provide a deterministic model for one entry, optional priors for shared parameters, and an observation likelihood. BET handles population training, new-entry prediction, posterior sample bookkeeping, and scoring. It uses JAX to evaluate the model across entries and posterior draws, so model functions must use JAX-compatible operations and avoid side effects.

An entry is one independently indexed unit (for example, a patient, experiment, or simulated replicate). Arrays share a leading entry axis. Observations within an entry, such as repeated measurements over time, belong on trailing axes and are modeled jointly by that entry's likelihood. Treating repeated measurements as separate entries would change the hierarchical interpretation.

Shared parameters and local z-scores. Global parameters are shared across training entries; local latent effects vary by entry and have standard-normal marginal priors. For example, a model might define a positive rate as `rate_i = exp(log_rate_mean + log_rate_sd * z_i)`. Here `z_i` is a z-score on the log-rate scale, while `rate_i` is lognormally distributed on its physical scale. The location and scale parameters are supplied by the user model or inferred through `GlobalPriorFn`; BET does not automatically invent a mean and scale for every local variable.

When there are multiple local variables, their joint distribution can be correlated through the learned Cholesky factor. A positive correlation means that entries with unusually high values of one standardized effect tend also to have high values of another, **before conditioning on their individual observations**. Posterior local values conditioned on data need not have sample mean zero or standard deviation one.

Training and testing serve different purposes. Training uses selected entries to estimate shared parameters, correlations, and training-specific local effects. Testing uses the learned population state to predict outcomes for unseen entries. Without a test likelihood, local effects are sampled from the learned population distribution. With a test likelihood and an inference Method, the local effects are conditioned on the new entry's available observations. A test set must not be included in training if the goal is out-of-sample evaluation.

Designing a BET model

1. Prepare `data` as a dictionary of equally long first-axis arrays. Decide which trailing axes represent time, features, or spatial positions.
2. Implement `ModelFn(globalParams, localZs, data)` for **one entry**. Return named arrays or scalars in a dictionary. Do not sample NumPyro random variables in this deterministic model function.
3. Implement `GlobalPriorFn(dataShapes)` when shared parameters must be inferred. Return NumPyro distribution objects keyed by unique parameter names. If there are no shared unknown parameters, omit the function.
4. Specify `localZNames` only for genuine entry-varying effects. Map those standardized effects to physical parameters inside `ModelFn`.

5. Implement the training likelihood as a dictionary of `(observation, distribution)` pairs. Use distributions with correct event and batch shapes; use explicit masks for absent observations.
6. Select training indices and a configured `Nuts(...)` or `Svi(...)` instance. Call `Train`, inspect diagnostics, and verify model behavior with posterior predictive checks.
7. Select held-out indices. Call `Test` without a likelihood for population prediction, or provide a test likelihood and Method for inference using partial observations.
8. Define a score function returning a scalar or array for one entry and evaluate `ScoreTrainSamples` or `ScoreTestSamples`. Summarize over posterior axes deliberately; do not confuse draw axes with entry axes.

A useful design rule is that **data determine the observation model, not the mechanistic model's parameter names**. Baseline covariates can enter `ModelFn` or global-parameter transformations, while observation masks determine which outcomes are used for inference.

Part 2: Complete worked example

End-to-end example: predicting longitudinal growth

This tutorial connects population training, predictions for unseen individuals, and updating those predictions after observing the beginning of each individual's trajectory. It uses synthetic exponential-growth data so that no external dataset is required. The goal is to compare **population-only prediction** against **prediction conditioned on three measurements**, using the same trained population model.

Generate data and define the model

```
import jax.numpy as jnp
from jax import random
import numpyro.distributions as dist
from BayesianEstimationTool import Estimator, Svi

n_people = 10
times = jnp.linspace(0.0, 7.0, 8)
key_g, key_noise = random.split(random.PRNGKey(42))
true_g = 0.15 + 0.025 * random.normal(key_g, (n_people,))
true_y = jnp.exp(true_g[:, None] * times[None, :])
y = true_y + 0.05 * random.normal(key_noise, true_y.shape)
data = {"time": jnp.tile(times, (n_people, 1)), "y": y}

def global_prior(shapes):
    return {
        "g_mean": dist.Normal(0.15, 0.10),
        "g_sd": dist.HalfNormal(0.10),
        "noise": dist.HalfNormal(0.10),
    }

def model_fn(global_params, local_zs, data):
    g = global_params["g_mean"] + global_params["g_sd"] * local_zs["g"]
    return {"prediction": jnp.exp(g * data["time"])}

def train_likelihood(global_params, local_zs, data, model_out):
    return {"y": (data["y"], dist.Normal(model_out["prediction"], global_params["noise"]))}
```

`g_mean` and `g_sd` set the population center and spread of growth rates. Each individual's standardized `local_zs["g"]` becomes a physical growth rate through `g_mean + g_sd * z`. The separate `noise` parameter describes measurement variation, not biological heterogeneity.

Train on six individuals

```
est = Estimator(data, model_fn, localZNames=["g"],
                GlobalPriorFn=global_prior, rngSeed=123)
train_ids = list(range(6))
test_ids = list(range(6, 10))

est.Train(train_ids, train_likelihood,
          Svi(numSteps=1500, numSamples=300, progressBar=False))
```

The first six individuals train the population model; the other four are held out. The short optimization run is for demonstration only. In research applications, inspect convergence, posterior predictive behavior, and prior sensitivity.

Predict without seeing any test observations

First define a score for the **future** measurements, at indices 3 through 7. Scoring can use held-out outcomes even though inference must not use them.

```
def future_mae(model_out, data):
    return jnp.mean(jnp.abs(model_out["prediction"][3:] - data["y"][3:]))

est.Test(test_ids, nGlobalSamples=20, nLocalZSamples=100)
before = est.ScoreTestSamples(future_mae)
print("Population-only mean future MAE:", float(jnp.mean(before)))
```

Here BET samples 20 population posterior states and generates 100 local-effect draws for each state. None of the held-out individuals' observed `y` values inform their latent effects. The score array has shape `[20, 100, 4]`: global draw, local draw, and held-out individual.

Update predictions after three observations

Next, define a test likelihood that reveals only the first three measurements. Masking is important: the remaining measurements must not affect inference.

```
def test_likelihood(global_params, local_zs, data, model_out):
    observed = jnp.arange(data["y"].shape[0]) < 3
    prediction = model_out["prediction"]
    safe_y = jnp.where(observed, data["y"], prediction)
    return {
        "y": (safe_y, dist.Normal(prediction, global_params["noise"]).mask(observed))
    }

est.Test(test_ids, TestLikelihoodFn=test_likelihood,
         Method=Svi(numSteps=800, numSamples=100, progressBar=False),
         nGlobalSamples=20)
after = est.ScoreTestSamples(future_mae)
print("Conditioned mean future MAE:", float(jnp.mean(after)))
```

BET now infers each held-out individual's local growth effect from the first three observations, conditional on each sampled population state. The score still evaluates measurements at indices 3 through 7, which were excluded from the test likelihood. This second test call replaces the previous test state; the `before` scores remain available because they were already stored in a separate variable.

Interpret the comparison

Both `before` and `after` have shape `[20, 100, 4]`. Their means summarize future absolute prediction errors across population draws, local draws, and held-out individuals. Conditioning on early measurements often improves prediction, but an improvement is not guaranteed in every small synthetic experiment.

The two stages answer different questions:

- **Population-only:** What can we predict for a new individual before observing their trajectory?
- **Conditioned:** What can we predict after observing that individual's first three measurements?

To inspect uncertainty in predicted trajectories rather than uncertainty in errors, define a score function that returns `model_out["prediction"]`. Summarize the posterior draw axes to obtain intervals for the latent mean trajectory. These intervals do not automatically include new measurement noise. For a stronger evaluation, repeat the experiment over multiple random seeds and vary the number of early observations.

The same Train → Test → Score workflow also applies to differentiable mechanistic ODE models.

Part 3: Model components and hierarchical effects

Components and function contracts

`Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)` takes a dictionary of arrays. The first axis of each data array indexes individuals; all arrays have the same number of individuals. Remaining axes can represent times, features, or spatial coordinates.

`ModelFn(globalParams, localZs, data)` receives dictionaries of global parameters, one individual's local latent values, and one individual's data. It returns a dictionary of model outputs. BET vectorizes this function across individuals and posterior draws; avoid Python side effects and conversions of traced JAX values to NumPy.

`GlobalPriorFn(dataShapes)` returns a dictionary of parameter names mapped to NumPyro distribution objects. Each shape in `dataShapes` excludes the individual axis. The global parameters are shared by the training population.

`TrainLikelihoodFn(globalParams, localZs, data, modelOut)` and `TestLikelihoodFn` each return a dictionary of site names mapped to `(observed_value, distribution)` tuples. A site may represent one scalar or a vector of observations; observation, distribution, and mask shapes must agree.

`ScoreFn(modelOut, data)` returns a scalar or array for one individual. BET vectorizes it over posterior samples and individuals.

Hierarchical local variables: standardized z-scores

Local z-scores represent entry-to-entry differences on a standardized scale. For example, `z=0` gives the model's central growth rate; positive or negative values change it according to the transformation you define. With `rate = exp(log_rate_mean + log_rate_sd * z)`, `log_rate_mean` sets the center on the log scale and `log_rate_sd` sets its spread. Before conditioning on individual observations, each z-score has a standard-normal marginal distribution.

Extend the tumor-growth example by adding treatment. Each patient now has two parameters: a **growth rate**, describing how quickly the tumor grows without treatment, and a **treatment-response parameter**, describing how strongly treatment slows that growth. Both may vary between patients. For instance, patients whose tumors grow faster might also tend to respond more strongly to treatment, or the relationship might be reversed. Assuming the parameters vary independently would miss that population-level relationship.

Set `localZNames=["growth", "response"]` to represent both sources of patient-to-patient variation as standardized local effects. BET can learn their **correlation** from the training patients. A positive correlation means patients with above-average growth effects tend also to have above-average response effects; a negative correlation means higher growth effects tend to accompany lower response effects. These are correlations between the standardized effects, not necessarily between the parameters after arbitrary transformations.

To represent this dependence while keeping each local effect marginally standard normal, BET uses a **Cholesky factor** of a correlation matrix. A correlation matrix specifies how pairs of effects vary together; its Cholesky factor is a triangular matrix that can transform independent standard-normal draws into correlated ones. BET places an LKJ prior on the correlation structure, with `lkjConcentration` controlling how strongly the prior favors correlations near zero. The inferred relationship can improve predictions for a new patient: measurements that inform their growth rate can also inform their likely treatment response, even if response measurements are limited.

The model below illustrates the parameter transformation. It is a schematic treatment model, not a validated description of tumor dynamics:

```
def model_fn(global_params, local_zs, data):
    growth = jnp.exp(global_params["log_growth_mean"] + global_params["log_growth_sd"] *
local_zs["growth"])
    response = jnp.exp(global_params["log_response_mean"] + global_params["log_response_sd"] *
local_zs["response"])
    net_rate = growth - response * data["treatment"]
    return {"prediction": data["baseline"] * jnp.exp(net_rate * data["time"])}
```

Define the population means, scales, and transformations you need explicitly; BET does not create them automatically for each local variable.

Part 4: Training and inference methods

NUTS and SVI reference

`Nuts(numWarmup=2000, numSamples=2000, numChains=4, acceptProb=0.95, denseMass=False, medianSamples=50)` uses NumPyro's NUTS MCMC kernel with median-based initialization. `numWarmup` is the adaptation period; `numSamples` is the retained number of draws **per chain**; `numChains` controls chain count; `acceptProb` is the target acceptance probability; `denseMass` enables dense mass-matrix adaptation; and `medianSamples` controls the initialization strategy. Its reported summary includes effective sample size and R-hat. Inspect divergences, mixing, and posterior predictive adequacy rather than treating a successful run as validation.

`Svi(numSteps=10000, numSamples=2000, stepSize=0.01, numParticles=1, progressBar=True, medianSamples=50, multivariate=False)` optimizes a variational approximation with Adam and an ELBO objective. `numSteps` is the number of optimization updates; `numSamples` controls approximate posterior draws generated after optimization; `stepSize` is Adam's learning rate; `numParticles` controls Monte Carlo particles for the ELBO; and `medianSamples` controls initialization. `multivariate=False` uses `AutoNormal`; `True` uses `AutoMultivariateNormal`, which can represent posterior covariance but may be expensive in large hierarchical models.

Extending inference methods. NUTS and SVI are the built-in choices, not a closed list. Additional inference methods can be integrated through BET's Method interface; see `METHOD_AUTHORING.md` and `ValidateMethod`.

Method selection. NUTS is often preferable when reliable posterior uncertainty is central and runtime is manageable. SVI can be faster for large problems, but an apparently stable loss curve does not establish calibrated uncertainty. Compare posterior predictive behavior and uncertainty against NUTS or simulated ground truth when feasible. BET supports configured Method instances; using Method classes can reconstruct previously saved configuration when available.

Part 5: Testing, conditioning, and scoring

Training, testing, and score dimensions

`Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)` estimates parameters from the selected individuals using a `Nuts(...)` or `Svi(...)` method.

`Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)` supports population prediction for new individuals when no test likelihood is supplied, or local-effect inference conditional on a test likelihood and Method. **The current implementation requires at least one local Z for `Test`.** `nGlobalSamples=None` uses posterior medians; a positive value samples stored training draws. `nLocalZSamples` applies to the no-likelihood mode.

`ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[posterior_draw, train_individual, ...score_shape]`.

`ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` returns `[global_draw, local_draw, test_individual, ...score_shape]`. For local-free models, save model outputs if scoring a test run.

Interpreting test-time uncertainty

BET can represent at least two distinct sources of predictive uncertainty: uncertainty in population-level parameters and uncertainty in a new entry's local effects. The current `Test` implementation requires at least one named local Z, even though `Estimator` and `Train` can be used without local Zs. Measurement noise and uncertainty in model structure add further uncertainty through the likelihood and scientific model.

With `nGlobalSamples=None`, Test uses the posterior median for global parameters and the learned Cholesky state rather than propagating variation across the training posterior. This is a useful conditional prediction but can understate uncertainty in the population. With a positive `nGlobalSamples`, BET samples global/Cholesky states from the training posterior and performs test inference for each selected state. This can be computationally more expensive because the test inference is repeated for each selected global state.

When no test likelihood is supplied, local effects are drawn from the learned population distribution. When a likelihood is supplied, the test method updates local effects using the observations represented by that likelihood, conditional on the selected global state. If the scientific target is prediction after one or more measurements, ensure the likelihood contains only measurements available at that prediction time. Score held-out outcomes separately and do not use them for local inference.

A score array is not itself a posterior predictive interval. To report uncertainty for a prediction, retain posterior predictions or scores over draws and calculate summaries or quantiles along the correct draw axes. State explicitly whether population uncertainty was propagated and whether scores are absolute error, squared error, log score, or another metric.

Conditioning on partial observations: a concrete pattern

To predict a trajectory from its first few measurements, expose only those measurements through the test likelihood. Keep scoring separate so it can evaluate later, held-out observations.

```
def test_likelihood(global_params, local_zs, data, model_out):
    # Assumes each entry has observations on the same sorted time grid.
    n_observed = 3
    observed = jnp.arange(data["y"].shape[-1]) < n_observed
    prediction = model_out["prediction"]
    # Replace held-out values before likelihood evaluation. The mask controls
    # which finite values contribute to the log likelihood.
    safe_y = jnp.where(observed, data["y"], prediction)
    return {
        "y": (safe_y, dist.Normal(prediction, 0.2).mask(observed))
    }

est.Test(testDataIdxs, TestLikelihoodFn=test_likelihood,
         Method=Svi(numSteps=1000, numSamples=200, progressBar=False))
heldout_scores = est.ScoreTestSamples(score)
```

Here `testDataIdxs` must identify entries excluded from training. The example assumes `data["y"]` is a time series and `model_out["prediction"]` has the same shape. Adapt the mask to each entry's actual observation schedule when lengths differ. If observations are irregular or have different valid times, use a validity mask combined with the observation cutoff. Ensure the deterministic model does not use future outcomes as inputs, and make every intermediate calculation finite at padded positions.

This call conditions local effects on the first three measurements, conditional on the selected global parameter state. With `nGlobalSamples=None` (the default), BET uses posterior medians for global parameters and the Cholesky factor; it does not integrate over their posterior variation. Set `nGlobalSamples` to a positive integer to repeat local inference across sampled global posterior states. This can be expensive, because BET runs the test inference separately for each selected state.

Return values, sample axes, and state

The estimator retains the most recent training and test state. A new call to `Train` clears the prior test state. A new call to `Test` replaces the previous test state. Score functions operate on this stored state; they do not refit the model.

- `Train(...)` returns the inference method's output object. For NUTS this is NumPyro's MCMC object; for SVI it is the SVI result containing optimized parameters and losses. BET also stores the posterior samples it needs for later testing and scoring.
- `Test(...)` without a likelihood performs population prediction and returns the model-output dictionary when `saveModelOut=True`; otherwise its result is `None`. With a test likelihood, it returns a list containing one inference output per selected global posterior draw. If `nGlobalSamples=None`, BET uses one median global state, so the list has one element.
- `ScoreTrainSamples(...)` returns an array with shape `[posterior_draw, training_entry, ...score_shape]`.
- `ScoreTestSamples(...)` returns an array with shape `[global_draw, local_draw, test_entry, ...score_shape]`. In the no-likelihood test mode, there is one global state if medians are used; with sampled global states, the leading axis indexes those states. The local-draw axis represents variation among newly generated or inferred local effects.
- `Save(path)` writes a compressed NPZ file and returns no meaningful value. `Save(None)` returns an in-memory nested record. `Estimator.Load(path, ModelFn=None)` returns the stored record as a dictionary; supplying a model function reconstructs an Estimator.

The precise model-output shape depends on what `ModelFn` returns. Always write down expected axes before aggregating: average over posterior draws to summarize uncertainty, and over entries only when that matches the scientific question. Avoid

flattening global and local draws together without considering how they were generated.

Part 6: Real-world data and numerical modeling

Variable-length sequences and masking

Masking is useful whenever entries have different numbers of measurements, a sensor reading is missing, or only some measurements should be revealed to the model during testing. It distinguishes **stored array positions** from **observations allowed to contribute to inference**. In particular, a held-out measurement may be retained for later scoring while excluded from the test likelihood.

Pad individual time series to a common length `T`, storing `time`, `y`, and a boolean `valid` mask as `[N, T]` arrays. Padded entries are storage placeholders, not observations. For longitudinal measurements, repeating the last valid time and measurement can be safer than inserting zeros, particularly when a solver or transformation cannot accept zero. The correct padding depends on the model and solver: the padded time grid must still satisfy any ordering requirements, and masked positions must not affect the likelihood.

```
data = {
    "time": jnp.array([[0., 1., 2.], [0., 3., 3.]]),
    "y": jnp.array([[1.0, 1.2, 1.4], [0.9, 1.1, 1.1]]),
    "valid": jnp.array([[True, True, True], [True, True, False]]),
}
```

Use a masked NumPyro distribution and replace invalid observed values with finite, safe values before log-probability evaluation:

```
def likelihood(global_params, local_zs, data, model_out):
    valid = data["valid"]
    pred = model_out["prediction"]
    safe_y = jnp.where(valid, data["y"], pred)
    return {"y": (safe_y, dist.Normal(pred, 0.2).mask(valid))}
```

The model must still produce finite predictions at padded positions. A zero mask does not guarantee safety if an intermediate expression already contains `NaN` or infinity. Do not multiply a divergent log-probability by zero and assume its gradient is safe.

To hide additional data during testing, store a separate observation mask, such as `test_observed = valid & (time <= cutoff)`. The test likelihood uses this mask; scoring may use the original validity mask to evaluate held-out observations. Do not leak hidden measurements into the model inputs or preprocessing.

Numerical stability when masking

Scale observations and model outputs to reasonable ranges, and use likelihood scales reflecting measurement noise rather than arbitrary tiny numbers. Extremely large residuals or near-zero noise can produce unstable log-probabilities and gradients. Replace missing observations with safe values before masking, and ensure model outputs remain finite even at padded positions. Use `jax.debug.print`, `jax.numpy.isfinite`, and small synthetic examples to isolate shape or gradient errors.

For multiple individuals, check each axis and broadcasting rule explicitly. Test the model and likelihood on one individual before fitting the population. Use posterior predictive checks; a successful NUTS or SVI run does not guarantee a scientifically adequate model.

JAX differential equation solvers

The model function may call a JAX-compatible ODE solver such as `jax.experimental.ode.odeint`. Keep solver inputs JAX arrays, use differentiable operations, and return outputs at the observation times. The project's examples use `odeint`, which has worked more reliably with BET's batched architecture than Diffrax in the workflows tested. Solver compatibility still depends on the model; verify gradients and numerical behavior.

```
from jax.experimental.ode import odeint

def rhs(state, time, rate):
    return rate * state

def model_fn(global_params, local_zs, data):
    rate = jnp.exp(global_params["log_rate"] + local_zs["rate"])
    times = data["time"]  # sorted, finite, compatible with odeint
    trajectory = odeint(rhs, data["initial"], times, rate)
    return {"prediction": trajectory}
```

With padded time grids, ensure solver times satisfy the solver's ordering requirements. A common approach is to use a shared regular solver grid and interpolate to each individual's observation times. Beware duplicate or unsorted padded times.

Part 7: Persistence and extension

Persistence and custom methods

`est.Save("fit.npz", saveAllData=True)` saves inference state and optionally data. Restore with `Estimator.Load("fit.npz", ModelFn=model_fn, GlobalPriorFn=global_prior)` when data was saved, or provide `data=` when it was not. Supply the original model and prior functions again; they are not serialized as executable code.

Custom inference methods must implement `_Fit(ModelFn, rngKey, samplePrefixes)` and `_SaveArgs()`. Run `ValidateMethod(method)` before use. See `METHOD_AUTHORING.md` for the minimum viable implementation and optional features.

Part 8: Troubleshooting and API reference

Common mistakes and troubleshooting

Shape mismatch. Print shapes before tracing, and start with a single entry. All top-level data arrays need the same entry count; per-entry model outputs must agree with likelihood observation shapes. Broadcasting may silently produce a plausible but incorrect likelihood, so verify expected dimensions explicitly.

Local z-scores mistaken for physical parameters. A latent z-score has a standard-normal marginal prior, not the same units as a growth rate, drug sensitivity, or measurement scale. Make all transformations explicit and document their units.

Hidden test outcomes leaking into inference. Supplying a full `y` array in test data is not itself a leak if the test likelihood masks unavailable values and the mechanistic model does not read those outcomes. Check all preprocessing, features, and masks to ensure future measurements cannot affect prediction.

Missing data and NaNs. Masked observations still require finite values at points where JAX evaluates intermediate expressions. Use safe finite placeholders and keep ODE solver time grids valid even for padded observations.

Unexpectedly narrow intervals. Check whether `nGlobalSamples=None` has fixed population parameters at their posterior medians. That mode does not propagate global posterior uncertainty. Use positive `nGlobalSamples` when that uncertainty matters, and remember that approximate SVI posteriors may be underdispersed.

Poor convergence. Rescale observations, inspect priors and parameter identifiability, and check whether the mechanistic solver produces finite gradients. For NUTS, examine chain mixing and convergence diagnostics; for SVI, inspect the loss trajectory and sensitivity to initialization.

Save/Load mismatch. Restore the same model function, global prior function, data shapes, and compatible Method configuration. Python function objects are not serialized; names are metadata, not executable code.

Detailed public API reference

The signatures below correspond to the public exports `Estimator`, `Nuts`, `Svi`, and `ValidateMethod` and the public Estimator methods.

Estimator construction

```
Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)
```

- `data`: dictionary of arrays sharing a leading entry axis.
- `ModelFn`: deterministic per-entry model function returning a dictionary.
- `localZNames`: optional list of named standardized local effects.
- `GlobalPriorFn`: optional function returning named NumPyro distributions for shared parameters.
- `lkjConcentration`: concentration of the LKJ correlation prior when multiple local effects are used.
- `rngSeed`: optional reproducibility seed.

Train

```
Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)
```

- `trainDataIdxs`: selected training entries.
- `TrainLikelihoodFn`: per-entry likelihood factory.
- `Method`: inference Method instance or supported Method class.
- `printSummary`: request inference summary output.
- `rngSeed`: optional call-specific seed.
- `saveLocalZs`: retain local posterior draws for later use.
- `saveModelOut`: retain generated model outputs rather than recomputing them.

Test

```
Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)
```

- `testDataIdxs`: entries for population prediction or local updating.
- `TestLikelihoodFn`: omitted for unconditioned population prediction; provided for observation-conditioned inference.
- `Method`: inference Method for observation-conditioned testing; may reuse saved configuration.
- `nGlobalSamples`: `None` uses posterior median population state; a positive integer selects trained posterior states.

- `nLocalZSamples`: number of local draws in no-likelihood testing.
- `printSummary`, `rngSeed`, `saveModelOut`: reporting, reproducibility, and output retention.

Scoring

`ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` evaluates training posterior predictions, with a draw axis followed by training-entry axis and any score dimensions.

`ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` evaluates the most recent test state, with global-draw and local-draw axes followed by test-entry axis and any score dimensions. Do not collapse axes without checking which dimension represents which source of uncertainty.

Persistence

`Save(fileName, saveTrainLocalZs=True, saveTestLocalZs=True, saveTrainModelOut=False, saveTestModelOut=False, saveAllData=False)` writes a compressed state file, or returns a record if `fileName=None`. Saving data increases portability but also file size and potential data-sensitivity obligations. The save flags govern inclusion of local draws and model outputs.

`Estimator.Load(fileName, ModelFn, GlobalPriorFn=None, data=None)` reconstructs an estimator using explicitly supplied functions. Provide `data` unless it was included in the saved state. Passing `ModelFn=None` retrieves the saved record instead of reconstructing an executable estimator.

`ValidateMethod(method)` checks a custom Method's required interface; see `METHOD_AUTHORING.md` for implementation details. Public exports are defined in package `__init__.py`.

Part 9: Verification and testing

Verification and further reading

The repository's `TESTING.md` explains how to run automated tests, while `METHOD_AUTHORING.md` documents the custom inference interface. The included `Examples/` directory contains NUTS, SVI, and Save/Load scripts. The one-page `docs/Cheatsheet.md` is a compact companion, not a replacement for this manual.

Automated smoke and integration tests demonstrate that supported workflows execute; they do not by themselves establish parameter identifiability, calibrated credible intervals, or validity of a particular scientific model. For research claims, add simulation-based recovery and posterior predictive checks suited to the application.

Test strategy

The regression suite exercises NUTS, SVI, scoring, validation, and Save/Load. Scientific validation additionally requires synthetic hierarchical parameter recovery, population-to-new-individual transfer with varying observation counts, and numerical equivalence after persistence. See `TESTING.md` for the test invocation.